SIMATS ENGINEERING

ASSESSMENT TOOL 1: Expert System Development Project

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS **COURSE CODE: MLA0101**

COURSE OUTCOME COVERED

CO 5: *Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.*(BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100 **Time: 90 Mins**

No. of Questions: 1

Annexure A

Expert System Development Project

Code of Conduct:

I **Kruthika Sre S / 192525360** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Kruthika Sre

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Acquisition and Knowledge Representation	10%	
3. Production Rule / Knowledge Base Design	15%	
4. Prolog Implementation and Programming	15%	
5. Forward Chaining Implementation and Demonstration	10%	
6. Backward Chaining Implementation and Demonstration	10%	
7. Procedural and Non-Procedural Paradigm Analysis	10%	
8. Unification, Backtracking and Inference Accuracy	10%	
9. Testing, Results and System Demonstration	5%	
10. Documentation, GitHub Repository and Technical Presentation	5%	
Total	**100%**	

Signature of the Faculty Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

Tool: SWI-Prolog

Usage of Prolog: Students shall use **Prolog** to develop a rule-based expert system by representing domain knowledge as **facts and production rules**, and implementing reasoning through **forward and backward chaining**. The project should demonstrate Prolog's **declarative/non-procedural nature**, logical inference, rule matching, and automated conclusion generation.

S.No.	Scenario-Based Expert System Development Question
1	**Medical Diagnosis Expert System:** Develop a rule-based expert system that identifies possible diseases based on symptoms, patient observations, and basic clinical information. Represent domain knowledge using production rules and implement both forward and backward chaining to derive conclusions.
2	**Crop Disease Advisory System:** Develop an expert system that identifies possible crop diseases based on leaf symptoms, soil conditions, weather conditions, and visible plant characteristics. Construct suitable production rules and demonstrate diagnosis using forward and backward chaining.
3	**Automobile Fault Diagnosis System:** Design an expert system that identifies probable vehicle faults based on symptoms such as engine noise, starting problems, warning indicators, abnormal vibration, and reduced mileage. Implement the knowledge base using production rules and compare forward and backward reasoning.
4	**Student Academic Advisory System:** Develop an expert system that recommends suitable academic interventions based on attendance, internal marks, assignment performance, learning difficulties, and previous academic performance. Use production rules and demonstrate both forward and backward chaining.
5	**Cybersecurity Threat Diagnosis System:** Construct a rule-based expert system that identifies possible cybersecurity threats from events such as repeated login failures, unusual network traffic, suspicious file activity, and unauthorized access attempts. Implement production rules and demonstrate how forward and backward chaining arrive at a threat classification.

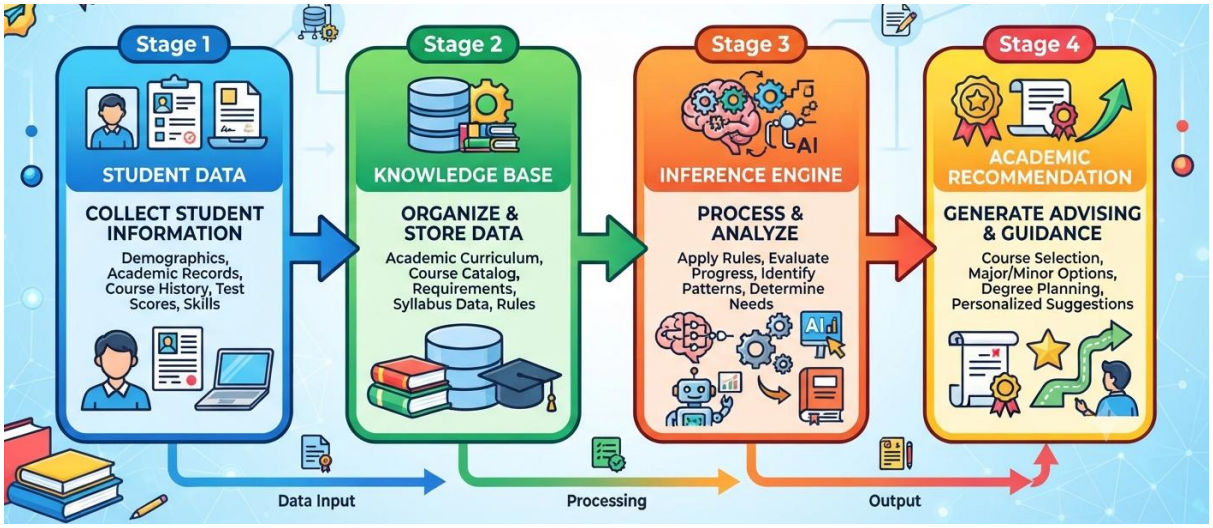
Report Format:

Stage	Student Activity	Expected Output
1. Domain Analysis	Identify the problem domain, entities, facts, symptoms/conditions, and possible conclusions.	Problem definition and domain model
2. Knowledge Acquisition	Collect domain knowledge from reliable sources or domain experts.	Knowledge specification
3. Knowledge Representation	Convert domain knowledge into IF–THEN production rules.	Knowledge base
4. Inference Design	Implement forward chaining and backward chaining.	Inference engine
5. Paradigm Analysis	Distinguish how the solution can be expressed using procedural and non-procedural approaches.	Paradigm comparison
6. System Development	Integrate the knowledge base, inference engine, user interface, and explanation facility.	Working expert system
7. Testing	Test the system using multiple facts and scenarios.	Test cases and outputs
8. Evaluation	Analyze correctness, coverage, consistency, and reasoning performance.	Evaluation results
9. Documentation	Prepare technical documentation and GitHub repository.	Project report + GitHub
10. Demonstration	Demonstrate the system and explain the reasoning process.	Project presentation/viva

4. STUDENT ACADEMIC ADVISORY SYSTEM

1. Domain Analysis

The Student Academic Advisory System focuses on identifying students who require academic support and recommending suitable interventions. The system considers factors such as attendance, internal marks, assignment performance, learning difficulties, and previous academic performance. Based on these factors, appropriate interventions such as academic counselling, remedial classes, additional assignments, mentoring, and study guidance are recommended.

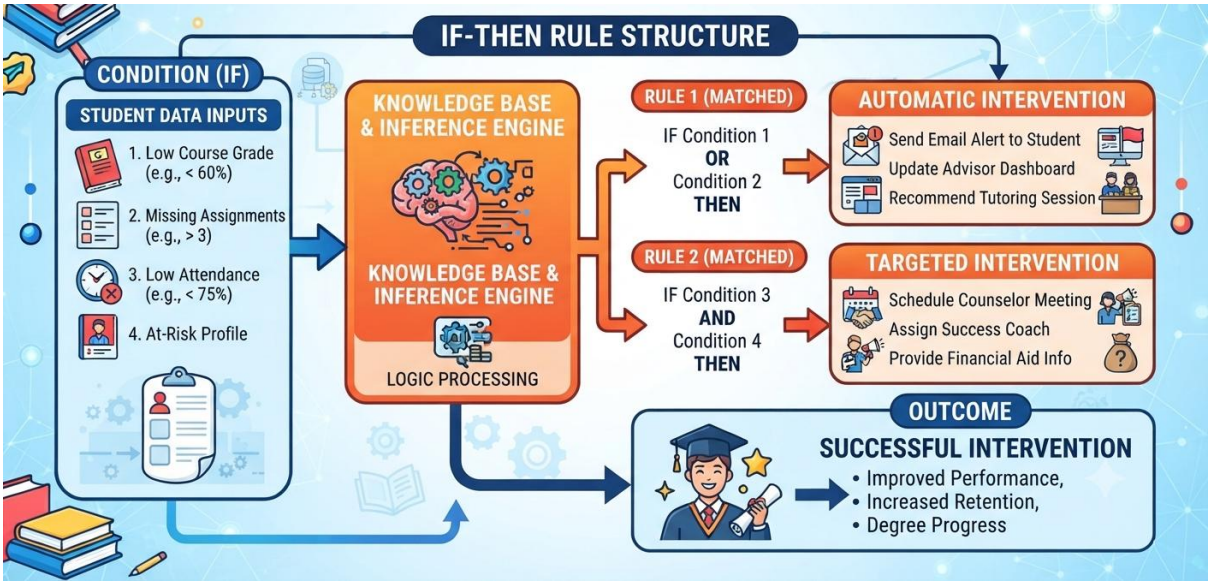


2. Knowledge Acquisition

Knowledge for the system is acquired from academic guidelines, teacher expertise, and commonly followed academic intervention practices. The collected knowledge describes relationships between student performance indicators and appropriate interventions. For example, consistently low internal marks may indicate the need for remedial academic support, while poor attendance may require attendance counselling.

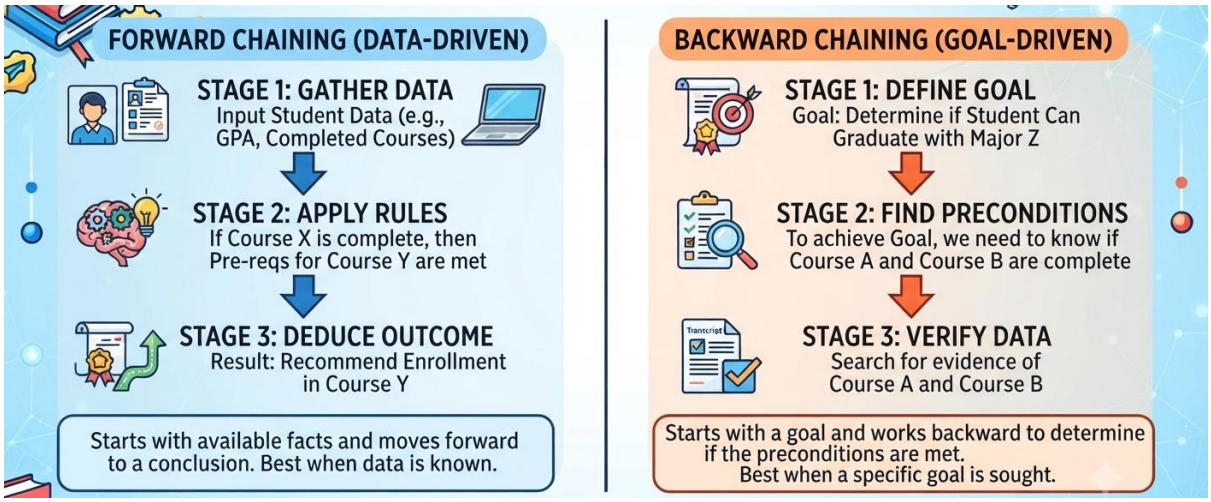
3. Knowledge Representation

The acquired knowledge is represented using production rules in the form of IF–THEN statements. These rules connect student academic conditions with appropriate recommendations. For example, a rule can be represented as: IF attendance is low AND internal marks are low THEN recommend academic counselling and remedial classes. This rule-based representation allows the system to make recommendations systematically.



4. Inference Design

The system uses both forward chaining and backward chaining for reasoning. In forward chaining, the system begins with known student information and applies relevant rules to derive an appropriate intervention. In backward chaining, the system begins with a possible recommendation and works backward to determine whether the student's conditions satisfy the rules required for that recommendation.



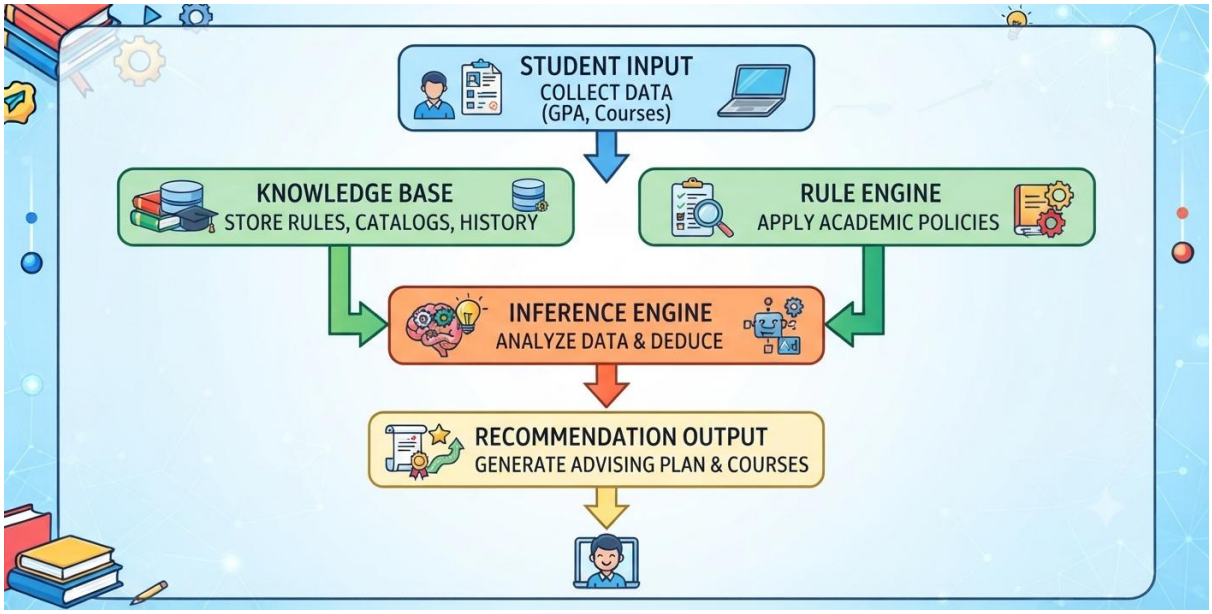
5. Paradigm Analysis

A rule-based expert system paradigm is selected because academic advisory decisions can be effectively represented using clearly defined conditions and recommendations. Production rules provide a transparent reasoning mechanism, allowing the recommendations generated by

the system to be easily understood and verified. Forward and backward chaining provide two different approaches for reaching the required conclusions.

6. System Development

The system is developed with major components including a student information input module, knowledge base, production-rule database, inference engine, and recommendation module. Student details such as attendance, marks, assignment performance, learning difficulties, and previous performance are provided as inputs. The inference engine evaluates these inputs against the knowledge base and produces suitable academic interventions.



7. Testing

The system is tested using different student academic scenarios. Test cases include students with low attendance, poor internal marks, weak assignment performance, learning difficulties, and consistently poor previous academic performance. The system is checked to ensure that the appropriate production rules are activated and that the resulting recommendations are correct.

8. Evaluation

The system is evaluated based on the accuracy and consistency of its recommendations and the correctness of its inference process. The results obtained through forward and backward chaining are compared with the expected recommendations. The evaluation determines whether the system can effectively identify academic difficulties and provide appropriate interventions.

9. Documentation

The system documentation includes the problem statement, objectives, domain analysis, knowledge acquisition process, production rules, knowledge representation, inference mechanisms, system design, implementation details, test cases, results, and evaluation. Examples of forward and backward chaining are also documented to clearly demonstrate the reasoning process.

10. Demonstration

The completed Student Academic Advisory System is demonstrated using sample student academic records. The demonstration shows how the system processes student information, applies the production rules, and generates appropriate academic interventions. Both forward chaining and backward chaining are demonstrated using suitable student cases to verify the expert system's reasoning capabilities.

EVALUATION RUBRICS:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Domain Understanding	10%	9–10% – Clearly analyzes the real-world problem, domain entities, facts, conditions, goals, and expected conclusions.	7–8% – Good understanding of the domain with minor omissions.	5–6% – Basic domain understanding with several missing elements.	0–4% – Poor or incorrect understanding of the problem domain.
2. Knowledge Acquisition and Representation	10%	9–10% – Acquires relevant domain knowledge and represents it accurately using well-structured Prolog facts and rules.	7–8% – Knowledge is appropriately represented with minor gaps.	5–6% – Basic knowledge representation with missing or unclear facts/rules.	0–4% – Inadequate or inappropriate knowledge representation.
3. Production Rule / Knowledge Base Design	15%	13–15% – Develops a comprehensive, consistent, and logically structured production-rule knowledge base with appropriate facts and rules.	10–12% – Well-designed knowledge base with minor inconsistencies or omissions.	7–9% – Basic rule base developed but several rules are incomplete or unclear.	0–6% – Knowledge base is incomplete, inconsistent, or inappropriate.
4. Prolog Implementation	15%	13–15% – Implements a fully functional expert system using Prolog facts, predicates, rules, variables, unification, and backtracking effectively.	10–12% – Functional Prolog implementation with minor technical issues.	7–9% – Basic implementation works with limited use of Prolog features.	0–6% – Implementation is incomplete, incorrect, or non-functional.

5. Forward Chaining	10%	9–10% – Correctly implements and demonstrates forward chaining from initial facts through applicable rules to final conclusions.	7–8% – Forward chaining works correctly with minor limitations.	5–6% – Basic forward reasoning demonstrated but with incomplete rule processing.	0–4% – Forward chaining is missing or incorrectly implemented.
6. Backward Chaining	10%	9–10% – Effectively demonstrates goal-driven backward chaining using Prolog's inference mechanism and clearly traces the reasoning process.	7–8% – Backward chaining works correctly with minor gaps.	5–6% – Basic backward reasoning demonstrated with limited explanation.	0–4% – Backward chaining is missing or incorrectly implemented.
7. Procedural and Non-Procedural Paradigm Analysis	10%	9–10% – Clearly distinguishes procedural and declarative/non-procedural approaches and demonstrates their application in the Prolog system.	7–8% – Provides a good distinction with minor conceptual gaps.	5–6% – Basic distinction is explained but lacks practical demonstration.	0–4% – Paradigms are misunderstood or not addressed.
8. Inference, Unification and Reasoning Accuracy	10%	9–10% – Produces accurate conclusions using unification, backtracking, and logical inference across diverse test cases.	7–8% – Reasoning is mostly accurate with minor errors.	5–6% – Basic inference works but produces occasional incorrect results.	0–4% – Inference is unreliable or incorrect.

9. Testing, Results and System Demonstration	5%	5% – Provides comprehensive test cases, accurate outputs, reasoning traces, and a convincing live demonstration.	4% – Good testing and demonstration with minor gaps.	2–3% – Limited test cases and basic demonstration.	0–1% – Testing or demonstration is missing/inadequate.
10. Documentation, GitHub and Technical Presentation	5%	5% – Complete documentation, well-organized GitHub repository, clear code, references, and professional presentation.	4% – Good documentation and repository with minor issues.	2–3% – Basic documentation/repository with several omissions.	0–1% – Poor documentation, missing repository, or inadequate presentation.
Total	100%				